



UNIVERSITI TEKNOLOGI MALAYSIA
FINAL EXAM
SEMESTER II 2024/2025

CODE OF SUBJECT : SECJ 4383
NAME OF SUBJECT : SOFTWARE CONSTRUCTION
ENGINEERING
YEAR / COURSE : 2/3/SECJ/3SCSJ
TIME : 3-Hours
DATE :
VENUE :

INSTRUCTIONS TO THE STUDENTS:

PART A: 4 Structure Questions

PART B: 1 Essay Question

Answer all questions in **PART A** and **PART B**.

Name	
I/C number	
Year/Course	
Section	
Lecturer	

THIS EXAMINATION PAPER CONSISTS OF 8 PRINTED PAGES INCLUDING THIS PAGE

PART A: (60 MARKS)

QUESTION 1 [15 marks]

BASED ON THE GIVEN CASE STUDY IN APPENDIX A, ANSWER THE QUESTIONS BELOW

- a. Draw a component diagram based on the given case study in Appendix A. [9 marks]
- b. Explain the **THREE (3)** common advantages associated with implementing inversion of control in software projects? [6 marks]

QUESTION 2 [15 marks]

ANSWER THE QUESTIONS BELOW

- a. Applying the Dependency Injection concept of the Spring Framework, draw a class diagram for an Online Shopping System. The system should involve classes for *Customer*, *Product*, *Cart*, and *Order*. Each *Product* object represents an item available for purchase, with attributes such as name, price, and quantity in stock. *Customers* can add products to their carts and place orders. The *Cart* class manages the items added by the customer for purchase, and the *Order* class handles the checkout process. Ensure that each customer has at least three different products in their cart during a shopping session. [6 mark]
- b. Analyze how dependency injection enables loose coupling and enhances the testability and maintainability of applications built on top of the Spring framework. [5 mark]
- c. Convert the *Customer* class based on the class diagram in Q2(a) into the standard JAVA and JSON representation using the Spring Framework. Include the necessary annotations **ONLY**. [4 marks]

QUESTION 3 [15 marks]

ANSWER THE QUESTIONS BELOW

Company ABC runs on a legacy inventory management system developed 15 years ago. Over the past few years, the system has shown signs of instability, is difficult to maintain, and cannot support recent business expansions such as e-commerce integration and AI-Based automation workflow. You are assigned to lead a proposal on whether to **reengineer the system** or build a new one from scratch.

Figure 3.1: Case Study 1

- a. You have been tasked with reengineering a legacy system in your organization. The system is outdated, has poor performance, and lacks scalability.
- i. Evaluate the decision to re-engineer the legacy system instead of build from scratch/replacing.
 - ii. Provide **THREE (3)** key benefits that can be achieved through the reengineering process.

[6 marks]

- b. Answer the questions based on the given case study in Figure 3.2 and Figure 3.3

Scenario:

You are working on a legacy Java application that manages customer orders. One of the methods, `processOrder()`, handles multiple tasks such as validating the order, calculating the total price, applying discounts, and printing the invoice. This method has grown lengthy and is challenging to maintain.

Figure 3.2: Case Study 2

- i. Identify the suitable code refactoring technique and discuss how the chosen technique can improve the code's maintainability and readability. **[3 marks]**
- ii. Identify two specific code segments within `processOrder()` that would benefit most from extraction into separate methods and provides a justification and sample possible methods. **[2 marks]**

```

1 public void processOrder(Order order) {
2     // Validate order
3     if (order.isValid()) {
4         // Calculate total price
5         double total = 0;
6         for (Item item : order.getItems()) {
7             total += item.getPrice();
8         }
9         // Apply discount
10        if (order.hasDiscount()) {
11            total -= order.getDiscountAmount();
12        }
13        // Print invoice
14        System.out.println("Invoice:");
15        for (Item item : order.getItems()) {
16            System.out.println(item.getName() + ": $" + item.getPrice());
17        }
18        System.out.println("Total: $" + total);
19    }
20 }
21 // Order class

```

Figure 3.3: Code snippet

c. Analyze the following code snippet:

```

if ((userAge > 60 && hasChronicIllness) || (isPregnant && !hasAllergy)) {
    grantPriorityAccess();
}

```

- i. Based on the above code snippet, describe the refactoring technique “*Simplify Conditional Statements*” and explain how it can improve the readability and maintainability of code.

[2 marks]

- ii. Rewrite the code where the “*Simplify Conditional Statements*” technique can be applied, and demonstrate the steps involved in refactoring the code.

[2 marks]

QUESTION 4 [15 marks]

ANSWER THE QUESTIONS BELOW

You are reviewing a legacy codebase of E-Commerce System. Your team notice that multiple classes contain identical blocks of code for calculating discounts and generating customer greetings. These problems lead to inconsistencies and makes bug fixing harder.

Figure 4.1: Case Study 3

```
1 // Appears in both OnlineOrder and WalkInOrder classes
2 double discount = 0;
3 if (customer.getMembershipLevel().equals("GOLD")) {
4     discount = totalAmount * 0.1;
5 } else if (customer.getMembershipLevel().equals("SILVER")) {
6     discount = totalAmount * 0.05;
7 }
8 // This code snippet is used to calculate the discount based on the customer's membership level.
```

Figure 4.2: Code Snippet

- a. Based on **Figure 4.1 and 4.2**, identify the design smells and explain **two (2) negative impacts** it may have on software quality. **[4 marks]**
- b. Based on answer in 4(a), rewrite the code and explain the solution by using the design smells you identified in answer 4(a). **[6 marks]**
- c. In a E-Commerce system, both the `Order` and `Invoice` classes require sending an email to the customer. However, the `Customer` class only provides a basic method called `updateEmail()` to manage the email address. Currently, email sending logic is duplicated across different classes that need to notify customers, causing multiple changes in unrelated parts of the system whenever email-related logic changes.

Based on this statement, answer all the questions:

- i. Analyze why it leads to poor software design based on the code smell identified.

[2 marks]

- ii. Explain a suitable refactoring solution using the concept of delegation or wrapper classes.

[3 marks]

PART B: Problem Solving (40% Marks)

QUESTION 1

BASED ON THE GIVEN CASE STUDY IN APPENDIX A, ANSWER THE QUESTIONS BELOW

- a. Apply the technique of Noun-Verb analysis based on the given case study in Appendix A. Use the table format to list out all your answers

[8 marks]

NOUN-VERB ANALYSIS STEPS	YOUR ANSWER

- b. A team of developers has come out with a domain model as shown in Figure 5.1. Analyse **THREE(3)** problems that can occur if the domain model is used for application domain development.

[6 marks]

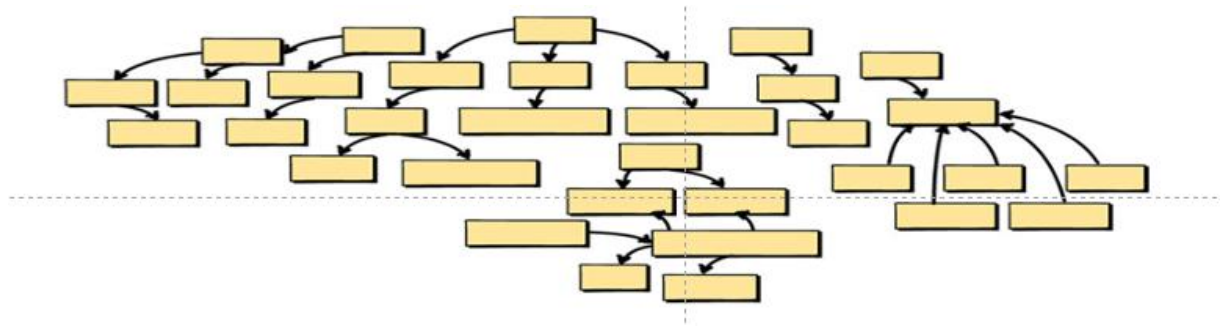


Figure 5.1 Domain Model

- c. Draw the Core Domain, Supporting Domains, and Generic Domain based on the analysis you have done in 1(a).

[12 marks]

- d. Based on Question 1(c):

- i) Draw the usage of **TWO (2)** different types of context mapping to integrate the different subdomains you have identified in 1(c).

[6 marks]

- ii) Justify the reasons you choose the two context mappings.

[4 marks]

- e. Based on Question 1(d):

- i) Identify **ONE (1)** reason why CRC cards are used in domain modeling.

[1 mark]

- ii) Apply the CRC template for **ONE (1)** class in your Core Domain.

[3 marks]

APPENDIX A: Case Study

NutriSense’s re-scoped software project now concentrates on five tightly knit components, each aimed at solving a single, high-impact need in the ready-to-eat (RTE) supply chain while exposing clean APIs for future extension.

Smart Vision QC sits at the factory gate. Edge cameras take high-resolution images of every incoming crate; a lightweight convolutional model grades size, colour, and visible defects in real time. The service returns a JSON verdict—*accept* or *reject*—and streams the decision to downstream services so only good-quality produce flows into production.

Shelf-Life Predictor picks up immediately after grading. It combines the QC score, precise temperature readings, and historical spoilage curves to forecast an exact “best-before” timestamp for each lot. Inventory staff receive automatic alerts when remaining shelf life falls below a configurable threshold, giving them time to reroute or discount at-risk stock.

DemandSense Forecaster looks outward. Pulling point-of-sale data, short-range weather forecasts, and social-media sentiment, it produces a rolling seven-day demand plan for every SKU. These forecasts drive the factory’s daily production schedule, ensuring NutriSense neither over-produces nor faces costly stock-outs.

Recipe Optimizer operates on the factory floor. Given a target nutrition label and the raw materials available—each stamped with its own shelf-life countdown—it proposes ingredient swaps that keep macronutrient values within legal tolerances while minimising waste. The optimiser’s decisions feed directly into the line’s portioning robots.

Traceability Ledger & IAM forms the project’s backbone. Every event—farm lot arrival, QC result, shelf-life prediction, recipe tweak, and shipment—lands in a graph database whose key facts are periodically hashed to a lightweight blockchain. An integrated identity-and-access-management layer issues OAuth tokens with strict role scopes, so farmers, planners, retailers, and regulators all see exactly—and only—the data they need. A consumer scanning the on-pack QR code therefore retrieves a tamper-evident, end-to-end history of the meal in under 200 milliseconds.

NutriSense gains a system that is complete enough to hit its freshness, waste-reduction, and transparency goals. Each component owns a single responsibility, communicates over well-defined interfaces, and can scale or be replaced independently as the business grows.

- END OF QUESTIONS -